12

Software Life Cycle

There must always be a discrepancy between concepts and reality, because the former are static while the latter is dynamic and flowing.
—Robert Pirsig, in Lila

A software life cycle model represents all the activities and work products necessary to develop a software system. Life cycle models enable managers and developers to deal with the complexity of the process of developing software in the same way as an analysis model or a system design model enables developers to deal with the complexity of a software system. In the case of software systems, the reality being modeled includes phenomena such as watches, accidents, trains, sensors, and buildings. In the case of software development, the reality includes phenomena such as participants, teams, activities, and work products. Many life cycle models have been published in the literature in attempt to better understand, measure, and control the development process. Life cycle models make the software development activities and their dependencies visible and manageable.

In this chapter, we describe selected well known life cycle models and revisit the activities described in previous chapters from the perspective of life cycle modeling. The modeling techniques we used for modeling software systems can also be used for representing life cycles. Whereas life cycle models are usually represented with ad-hoc notations, we use UML class diagrams and UML activity diagrams whenever possible. First, we describe the typical activities and work products of a software life cycle as defined by the IEEE standard 1074 [IEEE Std. 1074-1995]. Next, we introduce the Capability Maturity Model, a framework for assessing the maturity of organizations and their life cycles. Next, we discuss different life cycle models that have been proposed to deal with the development of complex software systems. We also introduce a new life cycle model called the issue-based model, where products and activities are modeled as a set of issues stored in a knowledge base. The issue-based model is an entity-centered view of the software life cycle that deals better with frequent changes within the duration of a project. We conclude with two examples of software life cycle models that can be used for a single semester or a two semester course project.

Software Life Cycle Page 458 Thursday, September 2, 1999 12:33 PM

12.1 Introduction

In Chapter 2, *Modeling with UML*, we describe modeling as classifying real world phenomena into concepts. So far, we have been interested in phenomena related to watches, emergency handling, facility management, train traffic control, vehicle part catalogs, and environmental modeling. This chapter, however, treats the software development process as the reality of interest. The phenomena we want to model include all the activities necessary to build a software system and the work products produced during development. Models of the software development process are called **software life cycle models**. Modeling the software life cycle is a difficult undertaking, because it is a complex and changing system. As software systems, software life cycles can be described by several different models. Most proposed life cycle models have focused on the activities of software development and represented them explicitly as first class objects. This view of the software life cycle is called **activity-centered**. An alternative view of the software life cycle is to focus on the work products created by these activities. This alternate view is called **entity-centered**. The activity-centered view leads participants to focus on how work products are created. The entity-centered view leads participants to focus on the content and structure of the work products.

Figure 12-1 depicts a simple life cycle for software development using three activities: `Problem definition`, `System development`, and `System operation`.

Figure 12-2 shows an activity-centered view of this simplistic life cycle model. The associations between the activities show a linear time dependency, which is implied by the use of the activity diagram notation: The problem statement precedes system development, which in turn precedes system operation. Alternate time dependencies are possible.

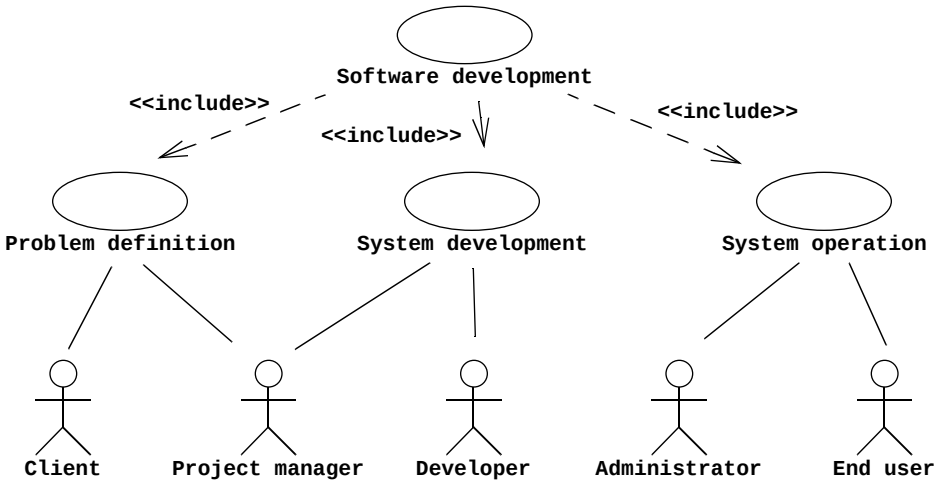

Figure 12-1 Simple life cycle for software development (UML use case diagram).

Software Life Cycle Page 459 Thursday, September 2, 1999 12:33 PM

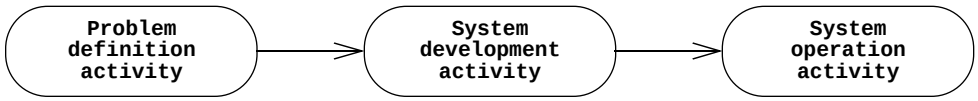

Figure 12-2 Simple life cycle for software development (UML activity diagram).

For example, in the software life cycle of Figure 12-3 the activities `System development` and `Market creation` can be done concurrently.

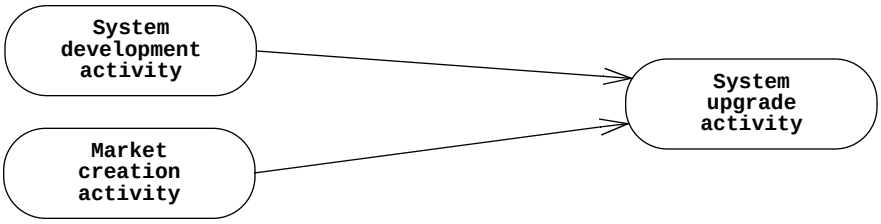

Figure 12-3 Another simple life cycle (UML activity diagram).

Figure 12-4 is an entity centered view of the model depicted by Figure 12-2. Software development produces four entities, a `Market survey document`, a `System specification document`, an `Executable system` and a `Lessons learned document`.

The activity-centered view and the entity-centered view are complementary, as illustrated by Figure 12-5. For every product there is one or more activity that generates it. The `Problem definition activity` uses a `Market survey document` as input and generates a `System specification document`. The `System development` activity takes the `System specification document` as input and produces an `Executable system`. `System operation` generates a `Lessons learned document` that could be used during the development of the next product. Alternatively, every activity generates one or more products.

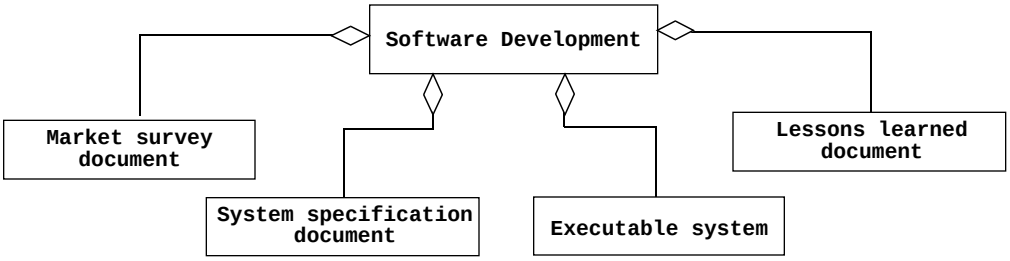

Figure 12-4 Appendix Figure 12-3 Entity-centered view of software development (UML class diagram).

Software Life Cycle Page 460 Thursday, September 2, 1999 12:33 PM

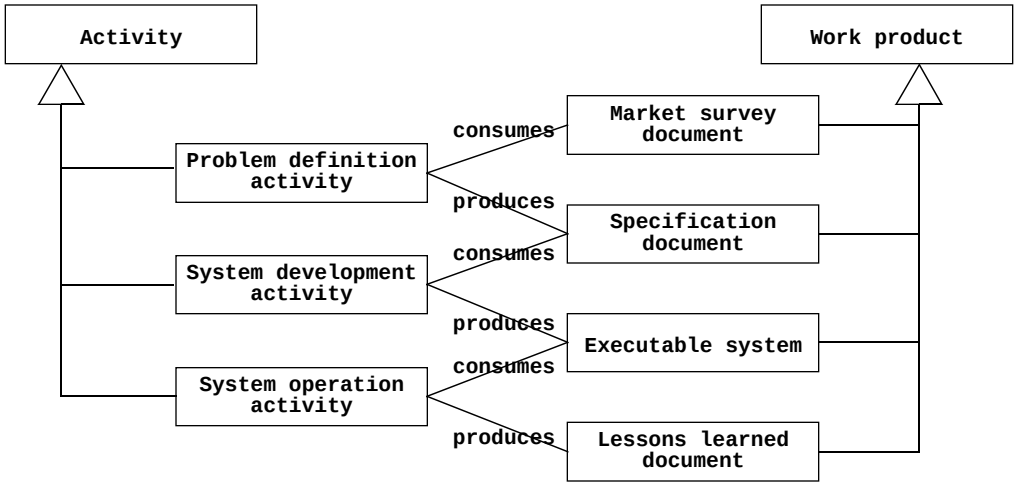

Figure 12-5 Activities and products of the simple life cycle of Figure 12-2 (UML class diagram).

In Section 12.2, we describe the life cycle activities defined by the IEEE 1074 standard [IEEE Std. 1074-1995]. This standard introduces precise definitions that enable project participants to understand and communicate effectively about the life cycle. In this section, we also describe the information flows between activities.

In Section 12.3, we describe the Capability Maturity Model, a framework for assessing the maturity of organizations and their life cycles. This framework enables organizations and projects to be compared, based on the activities of their life cycles.

In Section 12.4, we survey several activity-centered life cycle models that propose different ordering of activities. In particular, we discuss the waterfall model [Royse, 1970], the spiral model [Boehm, 1987], the V model [Jensen & Tonies, 1979], the sawtooth model [Rowen, 1990], and the Unified Software Process [Jacobson et al., 1999].

In Section 12.5, we describe two specific life cycle model instances for a team-based project, including its activities, products, and information flow.

12.2 IEEE 1074: Standard for developing life cycle processes

The *IEEE Standard for Software Life Cycle Processes* describes the set of activities and processes that are mandatory for the development and maintenance of software [IEEE Std. 1074-1995]. Its goal is to establish a common framework for developing life cycle models and provide examples for typical situations. In this section, we describe the main activities introduced by the standard and clarify its fundamental concepts using UML diagrams.

Software Life Cycle Page 461 Thursday, September 2, 1999 12:33 PM

12.2.1 Processes and activities

A **process** is a set of activities that is performed towards a specific purpose (e.g., requirements, management, delivery). The IEEE standard lists a total of 17 processes (Table 12-1). The processes are grouped into higher levels of abstractions called **process groups**. Examples of process groups are project management, pre-development, development, and post-development. Examples of processes in the development process group include:

- the *Requirements Process*, during which the developers develop the system models
- the *Design Process*, during which developers decompose the system into components
- the *Implementation Process*, during which developers realize each component

Table 12-1 Software processes in IEEE 1074

Process group	Processes
Life Cycle Modeling	Selection of a Life Cycle Model
Project Management	Project Initiation Project Monitoring and Control Software Quality Management
Pre-development	Concept Exploration System Allocation
Development	Requirements Design Implementation
Post-development	Installation Operation and Support Maintenance Retirement
Integral Processes	Verification and Validation Software Configuration Management Documentation Development Training

Each process is composed of activities. An **activity** is a task or group of sub activities that are assigned to a team or a project participant to achieve a specific purpose. The *Requirements Process*, for example, is composed of three activities:

- *Define and Develop Software Requirements*, during which the functionality of the system is defined precisely
- *Define Interface Requirements*, during which the interactions between the system and the user are precisely defined

Software Life Cycle Page 462 Thursday, September 2, 1999 12:33 PM

• *Prioritize and Integrate Software Requirements*, during which all requirements are integrated for consistency, and prioritized by client preference

Tasks consume resources (e.g., personnel, time, money) and produce a work product. During planning, activities are decomposed into project specific tasks, are given a start and ending date and assigned to a team or a project participant (Figure 12-6). During the project, actual work is tracked against planned tasks, and resources are reallocated to respond to problems.

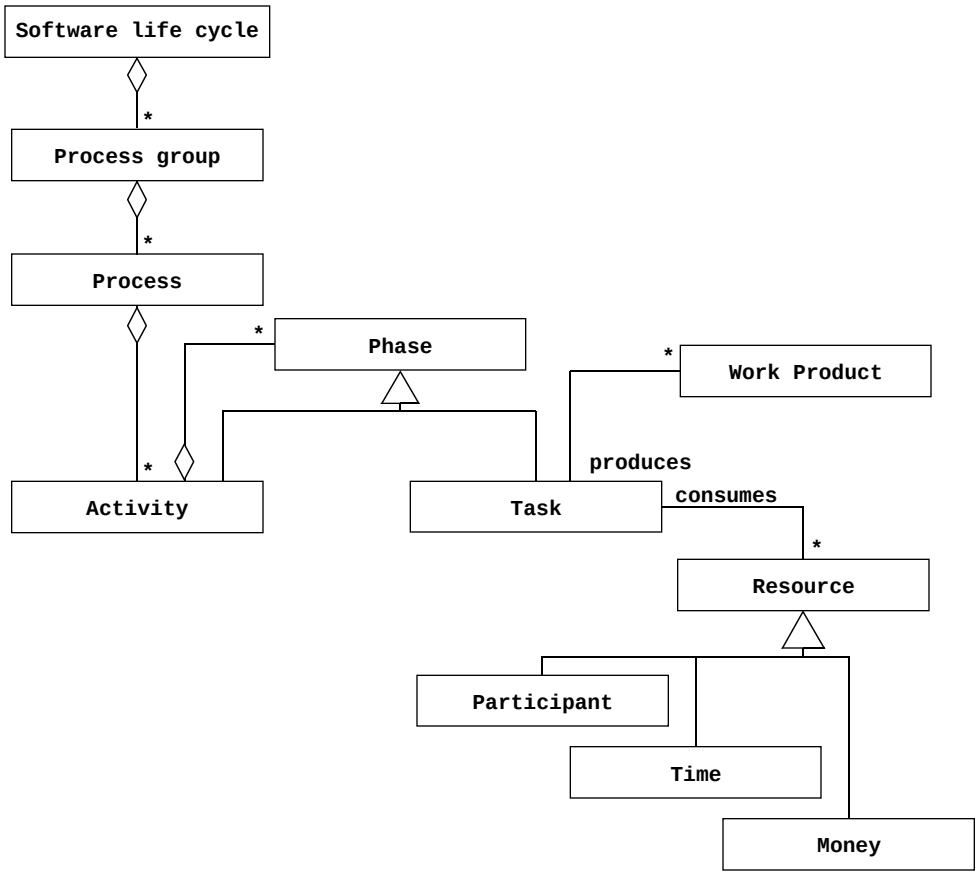

Figure 12-6 Model of the software life cycle (UML class diagram). A software life cycle consists of process groups, which in turn consist of processes. A process accomplishes a specific purpose (e.g., requirements, design, installation). A process consists of activities, which are in turn consists of sub - activities or tasks. Tasks represent the smallest piece of work that is relevant to management. Tasks consume resources and produces one or more work products. A project is an instance of a software life cycle.

Software Life Cycle Page 463 Thursday, September 2, 1999 12:33 PM

The processes required by IEEE 1074 are listed in Table 12-1. From the developers' point of view, the first six processes (i.e., the *Life Cycle Modeling Process*, the *Project Management Processes*, and the *Pre-development Processes*) have often already been initiated before their involvement with the project.

12.2.2 Life cycle modeling

During *Life Cycle Modeling*, the project manager customizes the activities defined in IEEE 1074 for a specific project (i.e., for an instance of a life cycle model). Not all projects require the same activities and the same sequencing of activities. For example, projects that do not deal with a persistent storage do not need to execute the activity *Design Data Base*. The selected life cycle model serves as input to the *Project Initiation Process* described in the next section. We provide two examples of *Life Cycle Modeling* in Section 12.5.

12.2.3 Project management

During the *Project Management* group the project manager initiates, monitors, and controls the project throughout the software life cycle. *Project Management* consists of three processes (Table 12-2).

The *Project Initiation Process* creates the infrastructure for the project. During this process the task plan, schedule, budget, organization, and project environment are defined. The project environment includes project standards, communication infrastructure, meeting and

Table 12-2 Project management processes

Process	Clause[a]	Activities
Project Initiation	3.1.3	Map Activities to Software Life Cycle Model
	3.1.4	Allocate Project Resources
	3.1.5	Establish Project Environment
	3.1.6	Plan Project Management
Project Monitoring and Control	3.2.3	Analyze Risks
	3.2.4	Perform Contingency Planning
	3.2.5	Manage the Project
	3.2.6	Retain Records
	3.2.7	Implement Problem Reporting Model
Software Quality Management	3.3.3	Plan Software Quality Management
	3.3.4	Define Metrics
	3.3.5	Manage Software Quality
	3.3.6	Identify Quality Improvement Needs

a. The 'Clause' column in this table and the other tables in this chapter is a clause number in IEEE 1074. This is a cross reference to the standards document as published in [IEEE Std. 1074-1995]

Software Life Cycle Page 464 Thursday, September 2, 1999 12:33 PM

reporting procedures, development methodology, and development tools. Most of the information generated during this process is documented in the *Software Project Management Plan (SPMP)*. The *Project Initiation Process* completes as soon as a stable environment for the project is established.

The *Project Monitoring and Control Process* ensures that the project is executed according to the task plan and budget. If the project manager observes any deviation from the schedule, she will take corrective action such as reallocating some of the resources, changing procedures, or replanning the schedule. The SPMP is updated to reflect any of these changes. The *Project Monitoring and Control Process* is active throughout the life cycle.

The *Software Quality Management Process* ensures that the system under construction meets the required quality standards (which were selected during Project Initiation). This process is executed by a separate quality management team to avoid conflicts of interest (i.e., the goal of the developers is to complete the system on time, the goal of the quality management team is to ensure that the system is not considered complete until it meets the required quality standard). The *Software Quality Management Process* is active throughout most of the life cycle.

We describe in Chapter 11, *Project Management* the activities of *Project Initiation* and *Project Monitoring and Control* that are related to planning, organization, and tracking. The activity *Establish Project Environment* requires particular attention in the context of a team-based project. One of the critical parts of the project environment is the communication infrastructure that will support the dissemination of information among the participants. To be able to react rapidly to changes and report problems without introducing an unreasonable overhead, all project participants need to be aware of the information flow through the project and the mechanisms for disseminating information. We describe in Chapter 3, *Project Communication* the activities related to defining and using the communication infrastructure. Note that in order to define the development team structure, and thus, the communication infrastructure, the initial system architecture (produced by the *System Allocation Process* described in the next section) needs to be defined.

12.2.4 Pre-development

During *Pre-development*, management (or marketing) and a client identify an idea or a need. This can be addressed with a new development effort (greenfield engineering), or a change to the interface of an existing system (interface engineering) or software replacement of an existing business process (reengineering). The *System Allocation Process* establishes the initial system architecture and identifies the hardware, software and operational requirements. Note that the subsystem decomposition is the foundation of the communication infrastructure among the project members. The requirements, subsystem decomposition and communication infrastructure are described in the *Problem Statement*[1], which serves as input for the *Development* process. The pre-development processes are depicted in Table 12-3.

Software Life Cycle Page 465 Thursday, September 2, 1999 12:33 PM

Table 12-3 Pre-development processes

Process	Clause	Activities
Concept Exploration	4.1.3	Identify Ideas or Needs
	4.1.4	Formulate Potential Approaches
	4.1.5	Conduct Feasibility Studies
	4.1.6	Plans System Transition (If Applicable)
	4.1.7	Refine and Finalize the Idea or Need
System Allocation	4.2.3	Analyze Functions
	4.2.4	Develop System Architecture
	4.2.5	Decompose System Requirements

12.2.5 Development

Development consists of the processes directed toward the construction of the system.

The *Requirements Process* starts with the informal description of the requirements and defines the system requirements in terms of high-level functional requirements, producing a complete specification of the system and a prioritization of the requirements. We describe the *Requirements Process* in Chapter 4, *Requirements Elicitation* and Chapter 5, *Analysis*.

The *Design Process* takes the architecture produced during the *System Allocation Process* and the specification from the *Requirements* and produces a coherent and well-organized representation of the system. The activities *Perform Architectural Design* and *Design Interfaces* activities result in a refinement of the subsystem decomposition. This also includes the allocation of requirements to hardware and software systems, the description of boundary conditions, the selection of off-the-shelf components and the definition of design goals. The detailed design of each subsystem is done during the *Perform Detailed Design* activity. The *Design Process* results in the definition of design objects, their attributes and operations, and their organization into packages. By the end of this activity, all methods and their type signatures are defined. New classes are introduced to take into account nonfunctional requirements and component specific details. The *Design* process used in this book is described in Chapter 6, *System Design*. and Chapter 7, *Object Design*.

The *Implementation Process* takes the design model and produces an equivalent executable representation. The *Implementation Process* includes integration planning, and integration activities. Note that tests performed during this process are independent of those performed during quality control or *Verification and Validation.* We describe in Chapter 9, *Testing* the testing and integration aspects of the *Implementation*. The development processes are depicted in Table 12-4.

1. The *Statement of Need* mentioned in the IEEE 1074 standard is similar to the *Problem Statement*, but does not contain any project organization information.

Software Life Cycle Page 466 Thursday, September 2, 1999 12:33 PM

Table 12-4 Development processes

Process	Clause	Activities
Requirements	5.1.3	Define and Develop Software Requirements
	5.1.4	Define Interface Requirements
	5.1.5	Prioritize and Integrate Software Requirements
Design	5.2.3	Perform Architectural Design
	5.2.4	Design Data Base (If Applicable)
	5.2.5	Design Interfaces
	5.2.6	Select or Develop Algorithms (If Applicable)
	5.2.7	Perform Detailed Design
Implementation	5.3.3	Create Test Data
	5.3.4	Create Source
	5.3.5	Generate Object Code
	5.3.6	Create Operating Documentation
	5.3.7	Plan Integration
	5.3.8	Perform Integration

12.2.6 Post-development

Post-development consists of the installation, maintenance, operation and support, and retirement processes (Table 12-5).

During *Installation* the system software is distributed and installed at the client site. The installation culminates with the client acceptance test according to the criteria defined in the *Project Agreement. Operation and Support* is concerned with user training and operation of the system. *Maintenance* is concerned with the resolution of software errors, faults and failures after the delivery of the system. *Maintenance* requires a ramping of the software life cycle processes and activities into a new project. *Retirement* removes an existing system terminating its operations or support. *Retirement* takes place when the system is upgraded or replaced by a new system. To ensure a smooth transition between the two systems, both systems are often run in parallel until the users have gotten used to the new system. Except for client delivery and acceptance, we do not address the post-development processes in this book.

Software Life Cycle Page 470 Thursday, September 2, 1999 12:33 PM

Table 12-7 Mapping process maturity level, key process areas and IEEE processes

Maturity Level	Key Process Area
Initial	Not applicable
Repeatable	Focus: Establish basic project management controls. • **Requirements Management:** Requirements are baselined in a project agreement and maintained during the project • **Project Planning and Tracking**: A software project management plan is established at the beginning of the project and is tracked during the execution of the project. • **Subcontractor Management:** The organization selects and effectively manages qualified software subcontractors. • **Quality Assurance Management:** All deliverables and process activities are reviewed and audited to verify that they comply with standards and guidelines adopted by the organization. • **Configuration Management:** A set of configuration management items is defined and maintained throughout the entire project.
Defined	Focus: Establish an infrastructure that allows a single effective software life cycle model across all projects. • **Organization process focus:** The organization has a permanent team to constantly maintain and improve the software life cycle. • **Organization process definition:** A standard software life cycle model is used for all projects in the organization. A database is used for software life cycle-related information and documentation. • **Training program:** The organization identifies the needs for training for specific projects and develops training programs. • **Integrated Software management:** Each project has the possibility to tailor their specific process from the standard process. • **Software product engineering:** The software is built in accordance with the defined software life cycle, methods and tools. • **Intergroup coordination:** The project teams interact with other teams to address requirements and issues. • **Peer reviews:** Developer examine deliverables on a peer level to identify potential defects and areas where changes are needed.

Software Life Cycle Page 471 Thursday, September 2, 1999 12:33 PM

Table 12-7 Mapping process maturity level, key process areas and IEEE processes (continued)

Maturity Level	Key Process Area
Managed	Focus: Quantitative understanding of the software life cycle process and deliverables. • **Quantitative process management:** Productivity and quality metrics are defined and constantly measured across the project. It is critical that these data are not immediately used during the project, in particular to assess the performance of developers, but are stored in a database to allow for comparison with other projects. • **Quality management:** The organization has defined a set of quality goals for software products. It monitors and adjusts the goals and products to deliver high-quality products to the user.
Optimized	Focus: Keep track of technology and process changes that may cause change in the system model or deliverables even during the duration of a project. • **Defect prevention:** Failures in past projects are analyzed, using data from the metrics database. If necessary, specific actions are taken to prevent those defects to occur again. • **Technology change management:** Technology enablers and innovations are constantly investigated and shared throughout the organization. • **Process change management:** The software process is constantly refined and changed to deal with inefficiencies identified by the software process metrics. Constant change is the norm, not the exception

12.4 Life cycle models

Figure 12-7 depicts the information flow among processes in the IEEE 1074 standard. The complexity of the standard is significant as can be seen in the many dependencies between the processes. Each association represents a work product that is generated by a process and consumed by another process. Each association also represents a formal communication channel between project participants supported by the exchange of documents, models, or code.

When selecting a life cycle model, a modeler has to address two questions: Is it necessary to model all these dependencies and in what order should they be scheduled? That is, which activities be left out and in what order should the remaining activities be performed and managed to deliver a high-quality system within budget and within time while change occurs during the duration of the project?

There is no single answer to these questions. First, different projects require different processes and different dependencies. For example, if the application domain is well known, as in the case of a business re-engineering project, the ordering of the development activities may be sequential, especially if the developers require only minimum training. A first of a kind project may require substantial prototyping in which case all the development processes should be executed concurrently. This, in turn, makes the management and configuration management processes much more complex.

Software Life Cycle Page 472 Thursday, September 2, 1999 12:33 PM

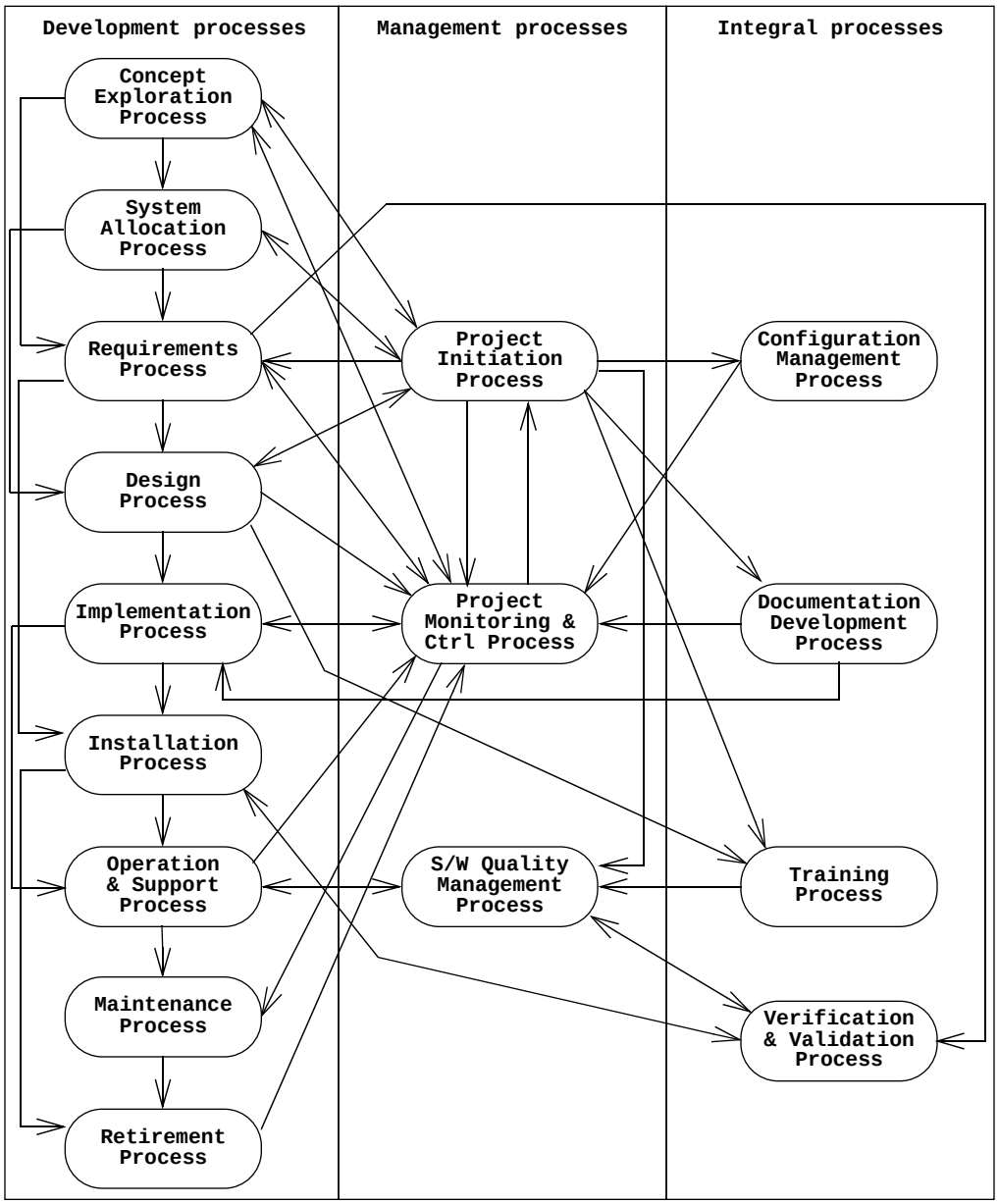

Figure 12-7 Process interrelationships in the IEEE 1074 standard (UML activity diagram, adapted from [IEEE Std. 1074-1995]]. As suggested by this picture, dependencies among processes and activities are complex and seldom allow a sequential execution of processes.

Software Life Cycle Page 473 Thursday, September 2, 1999 12:33 PM

The selection of the life cycle model also depends on the system model. Re-engineering a software system with existing use case and object models requires a different set of activities than building a system from scratch. In a project with experienced developers, good CASE tool support and a tight deadline, the project manager might select an entity-centered life cycle model instead of an activity-centered one.

For these reasons, IEEE 1074 does not dictate a specific life cycle model, but rather, provides a template that can be customized in many different ways. In this section we review selected life cycle models. Most of these models focus on the development processes exclusively.

12.4.1 Waterfall model

The **waterfall model** was first described by Royse [Royse, 1970]. The waterfall model is an activity-centered life cycle model that prescribes a sequential execution of a subset of the development processes and management processes described in the last section (Figure 12-8).

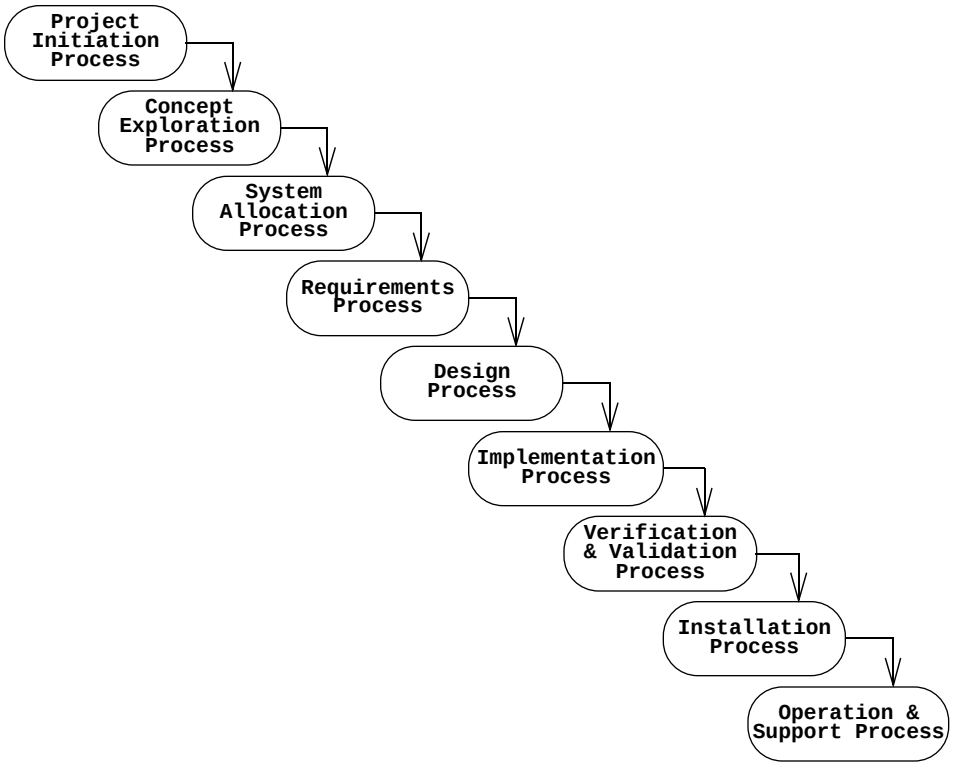

Figure 12-8 The waterfall model of software development is an activity-centered view of the software life cycle: Software development activities are performed in sequence (UML activity diagram adapted from [Royse, 1970] using IEEE 1074 names; project management and cross-development processes are omitted).

Software Life Cycle Page 474 Thursday, September 2, 1999 12:33 PM

The requirements activities are all completed before the system design activity starts. The goal is to never turn back once an activity is completed. The key feature of his model is the constant verification activity (called "verification step" by Royse) that ensures that each development activity does not introduce unwanted or deletes mandatory requirements. This model provides a simple (or even simplistic) view of software development that measures progress by the number of tasks that have been completed. The model assumes that software development can be scheduled as a step-by-step process that transforms user needs into code.

Figure 12-9 shows a widely used waterfall model, the DOD 2167A standard life cycle model.

The main characteristic of this model is that each development activity is followed by a review. The starting point in this model is the system requirements analysis activity whose goal is to generate unambiguous system requirements. The first review is the *System Requirements Review,* during which the requirements are reviewed with respect to completeness, consistency, and clarity. The system requirements are the basis for the system design activity, which generates the system design. The system design is reviewed during the *System Design Review* activity.

The system design is baselined once the system design review is successfully completed.The functional baseline serves as the starting point for the software requirements analysis, which creates the software requirements. The software requirements are then reviewed and baselined before serving as a basis for implementation. Implementation starts with the preliminary design followed by the detailed design activity. An important review is the critical design review (CDR). Coding does not occur before the CDR completes successfully.

12.4.2 V-model

The **V-model** is a variation of the waterfall model that makes explicit the dependency between development activities and verification activities. The difference between the waterfall model and the V model is that the latter makes explicit the notion of level of abstraction. All activities from requirements to implementation focus on building more and more detailed representation of the system, whereas all activities from implementation to operation focus on validating the system.

Higher levels of abstractions of the V-model deal with the requirements in terms of elicitation and operation. The middle-part of the V-model focuses on mapping the understanding of the problem into a software architecture. The lower level of the V-model focuses on details such as the assembly of software components and the coding of new ones. For example, the goal of the *Unit Test* activity is to validate units against their description in the detailed design. The *Component Integration and Test* activity validates functional components against the preliminary (or high-level) design.

Software Life Cycle Page 477 Thursday, September 2, 1999 12:33 PM

12.4.3 Boehm's spiral model

Boehm's spiral model [Boehm, 1987] is an activity-centered life cycle model that was devised to address the source of weaknesses in the waterfall model, in particular, to accommodate infrequent change during the software development. It based on the same activities as the waterfall model, however, it adds several activities such as risk management, reuse, and prototyping to each activity. These extended activities are called **cycles** or **rounds**.

The spiral model focuses on addressing risks incrementally, in order of priority. Each round is composed of four phases (Figure 12-11). During the first phase (upper left quadrant), developers explore alternatives, define constraints, and identify objectives. During the second phase (upper right quadrant), developers manage risks associated with the solutions defined during the first phase. During the third phase (lower right quadrant), developers realize and validate a prototype or the part of the system associated with the risks addressed in this round. The fourth phase (lower left quadrant) focuses on the planning of the next round based on the results produced in the current round. The last phase of the round is usually conducted as a review involving the project participants, including developers, clients, and users. This review covers the products developed during the previous and current rounds and the plans for the next round. Boehm's spiral model distinguishes the following rounds: *Concept of Operation*, *Software Requirements*, *Software Product Design*, *Detailed Design*, *Code*, *Unit Test*, *Integration and Test*, *Acceptance Test*, *Implementation*.[2]

Each round follows the waterfall model and includes the following activities:

1. determine objectives
2. specify constraints
3. generate alternatives
4. identify risks
5. resolve risks
6. develop and verify next level product
7. plan

The first two activities define the problem addressed by the current cycle. The third activity, *Generate Alternatives*, defines the solution space. The activities *Identify Risks* and *Resolve Risks* serve to identify future problems that may result in high costs or in the cancellation of the project. The activity *Develop and verify next level product* is the realization of the cycle, The activity *Plan* is a management activity preparing for the next cycle.

These rounds can be viewed in a polar-coordinate system shown in Figure 12-11. The first round, Concept of Operation, starts in the upper left quadrant. Subsequent rounds are

2. Note that the figure illustrates only the first three activities (Concept of Operation, Software Requirements and Software Product Design). The rounds for the remaining activities–Detailed Design, Code, Unit Test, Integration and Test, Acceptance Test–are not shown in detail, but only as blocks at the end of the last layer of the spiral

Software Life Cycle Page 478 Thursday, September 2, 1999 12:33 PM

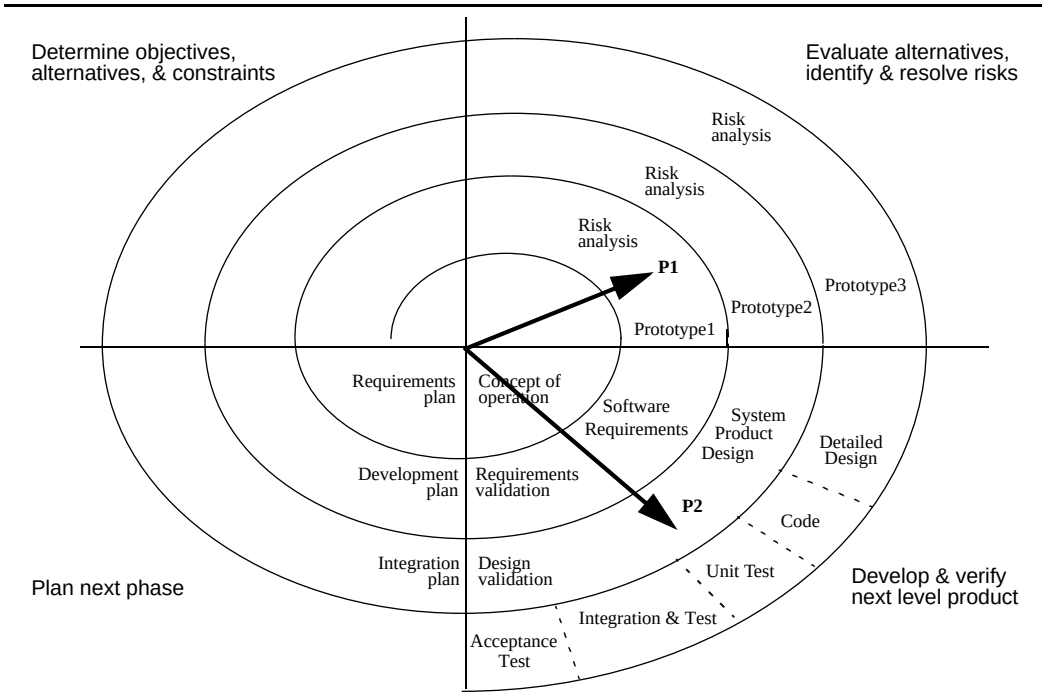

Figure 12-11 Boehm's spiral model (Adapted from [Boehm, 1987]). The distance from the origin represents the cost accumulated by the project. The angle from the horizontal represents the type of activity. For example, the project P1 is currently in the risk analysis activity associated with software requirements. The project P2 is in the development of the system product design.

represented as additional layers on the spiral. The notation allows easily to determine the status of the project over time. The distance from the origin is the cost accumulated by the project. The angular coordinate indicates the progress accomplished within each phase.

12.4.4 Sawtooth model

The life cycle models we described until now emphasize the management of software developers. They do not address the needs of the customer or the user. The waterfall model, the V model and the spiral model all assume that the software requirements will not change drastically during the duration of the project and that it is sufficient to show progress according to the development plan. The drawback of this approach is that the client and the user do not see a running system before the client acceptance test, and thus, cannot correct any requirement problems.

Users and implementors, however, have different needs when trying to understand software systems. The **sawtooth model** [Rowen, 1990] tries to solve this discrepancy by

Software Life Cycle Page 479 Thursday, September 2, 1999 12:33 PM

showing the user and software developer's perception of the system at different levels of abstractions over time.

At the beginning of a project, developers and the client are at the same level of abstraction, namely the requirements of the system as described in the problem statement. During development these points of view differ quite a bit. The user stays at the level of requirements, whereas the developers focus on feasibility. The software development process has to ensure that both points of view meet at the end of the project. The sawtooth model achieves this goal by introducing new activities. The span between these levels of abstractions corresponds to the gap between the user's perception of the system and the developer's perception of the system. To make sure that these meet at the end, checkpoints are introduced during development. This is usually done by getting the client involved *at their level of abstraction.*

For example, after the requirements and system design phase of an interactive software system, the developers can prototype the screen sequences in terms of the use cases describing the functional requirements. By demonstrating this prototype to the client, the client is able to evaluate quite early in the development whether the prototype satisfies the functional requirements. By repeating this process several times during the development, the manager makes sure that the trajectories intersect several times during the development. This makes it much more probable that they meet at the end of development.

The sawtooth model is a modified V-Model that includes these intersections. It is called sawtooth model, because each prototype demonstration results in a "tooth". The tip of each tooth is an intersection with the client's level of abstraction. Figure 12-12 shows the sawtooth model for a development project with two prototypes, a revolutionary prototype, and an evolutionary prototype.

The revolutionary prototype is often an illustrative prototype because it needs to be built quickly for demonstrating the functionality of the system. There is little intent to deliver this prototype for production use.[3] No matter how realistic, this prototype is still only a model of the system. The scenarios demonstrated will be contrived and represent only a small fraction of the required functionality. The short cuts taken to develop a quick version would be a maintenance nightmare if this prototype was promoted to production use.

The second prototype is usually an evolutionary prototype. It is shown late in the development where some functionality has already been implemented. The primary distinction between the two types of prototyping is that the revolutionary prototype does not need an overall design whereas the evolutionary prototype does. It is in general counterproductive to insist on a complete subsystem decomposition when showing the first prototype to the user.

The tasks and activities of the prototype development process in the sawtooth model are shown in Table 12-8.

3. If the revolutionary prototyping environment is identical to the development environment, then it might be possible to reuse some parts of the revolutionary prototype during implementation. But reuse is not the goal when producing a revolutionary prototype.

Software Life Cycle Page 480 Thursday, September 2, 1999 12:33 PM

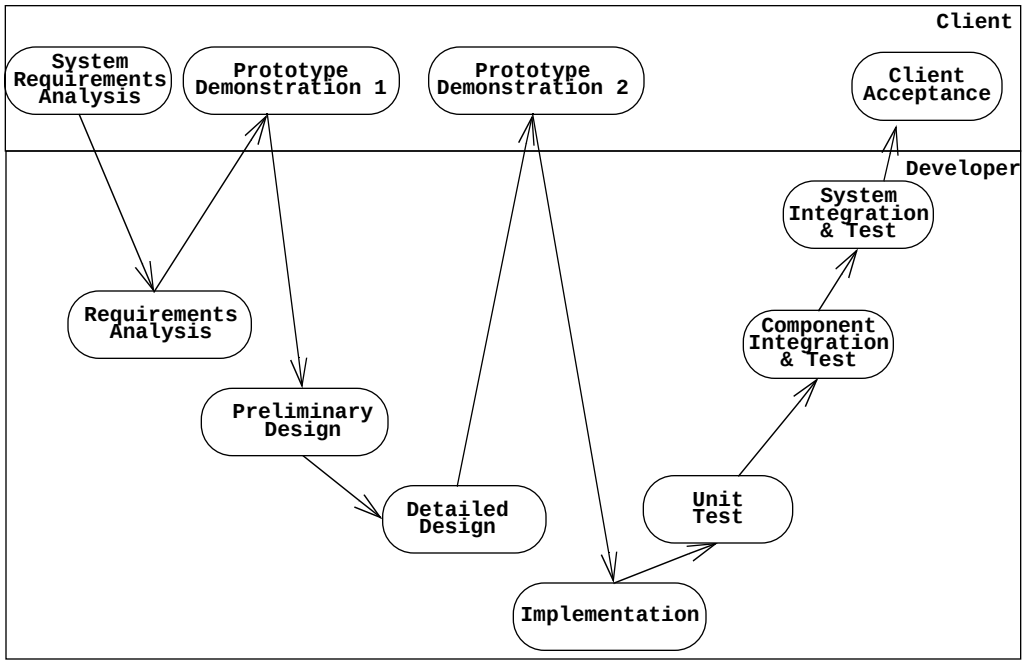

Figure 12-12 Sawtooth Model with 2 Prototype demonstrations (UML activity diagram). The Client swimlane encloses the activities that are visible to the client, whereas the developer swimlane encloses the activities that are at a lower abstraction level.

Table 12-8 Activities of the Prototype Development Process ("Tooth") in the Sawtooth Model

Activity	Task
Preparation	Select prototyping environment Select functionality to be demonstrated Develop prototype Develop agenda for prototype demonstration Notify client and review agenda
Demonstration	Install prototype Demonstrate selected functionality Record minutes
Evaluation of feedback	Review minutes from demonstration Identify issues Discuss issues
Project correction	Resolve open issues

Software Life Cycle Page 481 Thursday, September 2, 1999 12:33 PM

12.4.5 Shark tooth model

The **shark tooth model** is a refinement of the sawtooth model. In addition to the client demonstrations *(large teeth)* management reviews and demonstrations are introduced as well *(small teeth).*

For example, large teeth can include a functional prototype and a user interface mock-up. The former demonstrates the feasibility of the functions that will be implemented in the system, whereas the latter illustrates the layout (or look and feel) of the user interface. Small teeth can include a system integration prototype that demonstrates the interaction between the components of the system. Such an integration prototype can be built as soon as the components are selected and need not implement any functionality. The client is not a good target for the demonstration of a system integration prototype. In general the project manager is targeted as the audience for the system integration prototype demonstration and call the associated review an internal review. The system integration prototype can be demonstrated several times during the project, each of these demonstration leading to a "small tooth". To describe the demonstrations for management, we add another swimlane to the sawtooth model that depicts the level of understanding of the project manager (see Figure 12-13).

The project manager's swimlane is shown between the client and the developer swimlanes. Demonstrating the system integration prototype is a tooth involving the developers and the project manager. The shark tooth model assumes that the project manager is interested in system design and object design issues and thus wants to reach deeper level of system understanding than the client, and neither wants to follow the developer to the lowest levels of detail in the system. Small teeth are internal reviews involving a prototype demonstration for the project manager. Large teeth are prototype demonstrations for the client

12.4.6 Unified software development process

The **Unified Software Development Process** (also called the **Unified Process**) is a life cycle model proposed by Booch, Jacobson, and Rumbaugh [Jacobson et al., 1999]. Similar to Boehm's spiral model, a project consists of several cycles, each of which ends with the delivery of a product to the customer. Each cycle consists four phases: *Inception*, *Elaboration*, *Construction*, and *Transition.* Each phase itself consists of a number of iterations. Each iteration addresses a set of related use cases or mitigates some of the risks identified at the beginning of the iteration.

The inception phase corresponds to the IEEE 1074 *Concept Exploration* activity. During this phase, a need or an idea is defined and its feasibility is evaluated. The elaboration phase corresponds to the project initiation process, during which the project is planned, the system is defined and resources are allocated. The construction phase corresponds to the development processes. The transition phase corresponds to the installation and post-development processes.

The Unified Process emphasizes the staging of resources, an aspect of software development that is not captured in the other life cycle models. The Unified Process assumes that the activities *Requirements*, *Analysis*, *Design*, *Implementation* and *Testing* participate in

Software Life Cycle Page 482 Thursday, September 2, 1999 12:33 PM

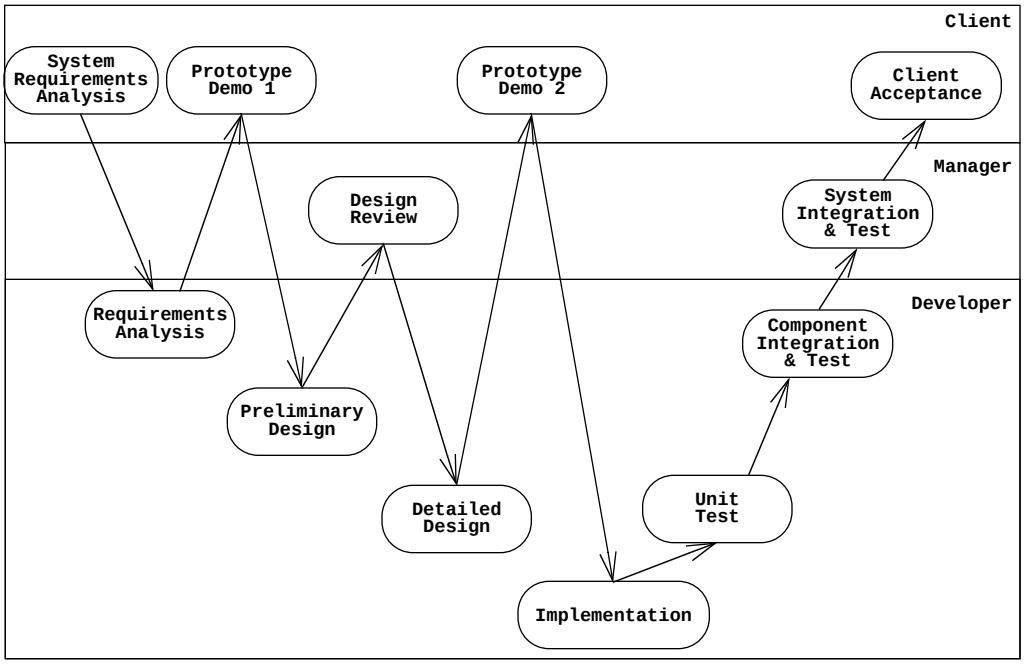

Figure 12-13 Shark tooth model with 2 Prototype demonstrations and 1 review (UML activity diagram, levels of abstraction are represented with swimlanes). Small teeth reaching the management swimlane are internal reviews involving a prototype demo by the developers for the project manager. Large teeth reaching the client swimlane are prototype demonstrations to the client.

each of these iterations. However, these activities have differing phase-specific needs. For example, during the elaboration phase, the requirements and analysis activities are allocated most of the resources. During the construction phase, the resource requirements for requirements and analysis activities diminish, but the design and implementation activities are allocated more resources.

Figure 12-14 shows an entity-centered view of the Unified Process as a set of models. The requirements are captured in the use case model. The analysis model describes the system as a set of classes. The design model defines the structure of the system as a set of subsystems and interfaces and the deployment model defines the distribution across. The implementation model maps the classes to components and the test model verifies that the executable system provides the functionality described in the use case model.

All models are related to each other through traceability dependencies. A model element can be traced to at least one element in an associated model. For example, every use case has a traceable relationship with at least one class in the analysis model. Traceability allows to understand the effect of change in one model on other models, in particular it allows to provide traceability in the requirements.

Software Life Cycle Page 483 Thursday, September 2, 1999 12:33 PM

The maintenance of the dependencies between the models in Figure 12-14 can be done in different ways. During **forward engineering**, analysis and design models are established from the use case model, and the implementation model and test models are then generated from these models. During **reverse engineering** the analysis and design models are extracted or updated from existing code. **Round trip engineering**, is a combination of reverse engineering and forward engineering. It allows the developer to switch between these development modes at any time, depending on which model is undergoing the most amount of change.

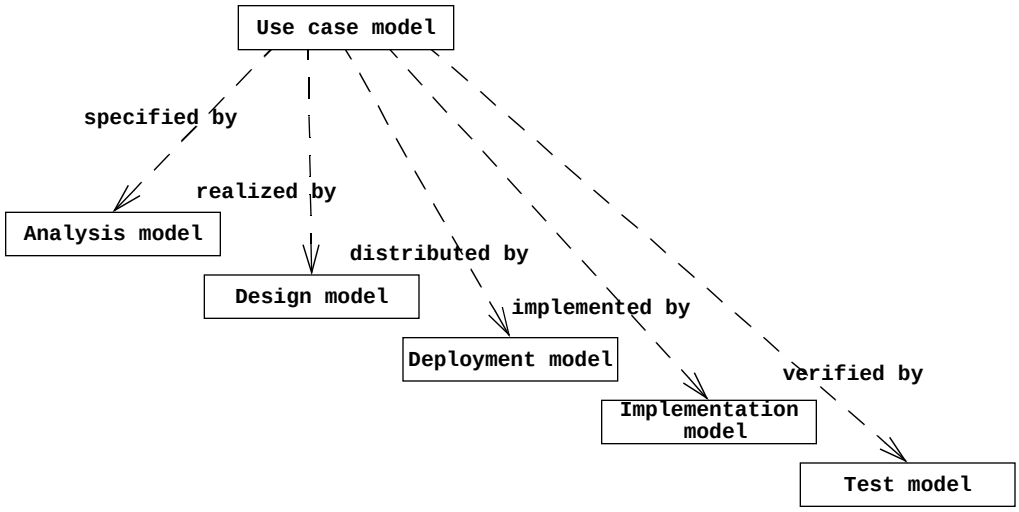

Figure 12-14 Entity-centered life cycle view of the models of the Unified Process (UML class diagram). the dependencies depict traceability. There are dependencies between all models. Only the dependencies between the use case model and the other models are shown [Jacobson et al., 1999].

12.4.7 Issue-based life cycle model

If the time between changes is significantly smaller than the duration of an activity, and if changes can occur in the application domain as well as in the solution domain, both the waterfall model and the spiral model exhibit problems. For example, assume the development team selects the hardware platform and that new platforms appear every three to four years. As long as the project time is significant less than 3 years, the knowledge of the development team at the beginning of the project is usually sufficient to select a platform once during system design. However, if a new platform appears every 3 to 4 months and the duration of the project is also 3 or 4 months, there is a high probability that the platform decision will need to be reconsidered during the project.

In the following we describe an entity-centered life cycle model, called the **issue-based life cycle model** that aims at dealing with frequent changes. This model is based on the rationale

Software Life Cycle Page 484 Thursday, September 2, 1999 12:33 PM

behind the system as an issue model (see *Chapter 8, Rationale Management).* Each project starts with a set of issues. If the project starts from scratch, these issues are drawn from the project manager's experience or from a standard template. In a re-engineering or interface project, issues may be available from the issue model of the previous project. If the project has a long history, the rationale should be well populated. Examples of issues are "How do we set up the initial teams?", "Should the mechanic have access to driver-specific information", "What is the appropriate middleware?" or "What software architecture shall we use?", "What should we use as implementation language"? All issues are stored in an issue base that is accessible to all project participants.

The status of an issue can be closed or open. A closed issue is an issue that has been resolved. For example, a closed issue can be a decision about the platform on which the system should run (e.g., Solaris). Closed issues can be reopened, however, as changes occur in the application or solution domain. For example, if we need to support additional platforms (e.g., Linux and Windows NT), we reopen the issue, reevaluate the alternatives, and provide a new resolution. Open issues are resolved by discussion and negotiation among project participants (see *Chapter 3, Project Communication).* An issue `i2` depends on another issue `i1` if the resolution of `i1` constrains the alternatives available for `i2`. Tracking the dependencies among issues enables developers to assess the impact of revisiting a given issue. The issue base also tracks dependencies among issues. Figure 12-15 shows a snapshot of the issue base of a project, depicting the issue status and dependencies.

Issues can be mapped to the activities of life cycle models described earlier. For example, assume the activities *Planning* and *System Design* are part of the selected life cycle model. "How do we set up the initial teams?" can be categorized as a planning issue and "What

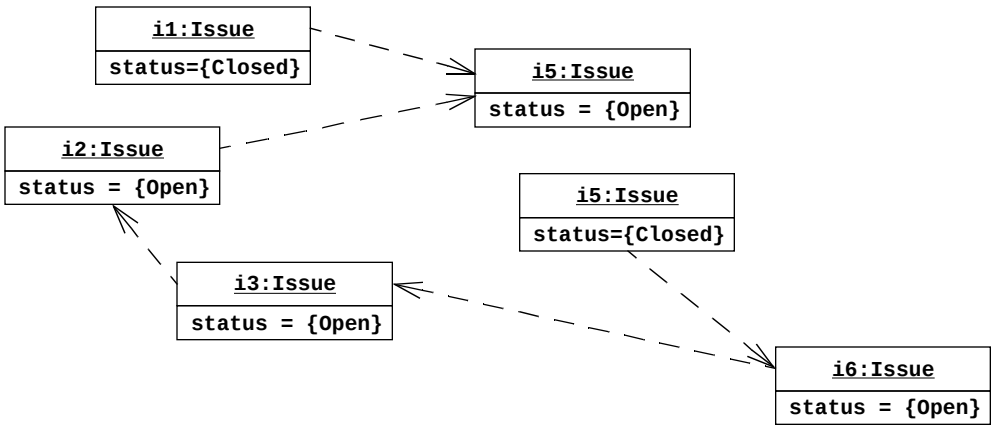

Figure 12-15 Snapshot of a project issue base (UML object diagram). Issues `i1` and `i5` have been resolved, whereas all other issues are still open. Dependencies among issues indicate that the resolution of an issue can constraint the alternatives for dependent issues.

Software Life Cycle Page 485 Thursday, September 2, 1999 12:33 PM

software architecture shall we use?" can be categorized as a system design issue. The status of issues can then be used to track the status of each activity. If there are any system design issues still open, the *System Design* activity has not yet been completed. The life cycle models we described earlier can then be seen as special cases of the issue-based model. In the Waterfall model, for example, developers completely resolve the issues associated with an activity before moving to the next activity. Figure 12-16 depicts the state of a project during *System Design*.

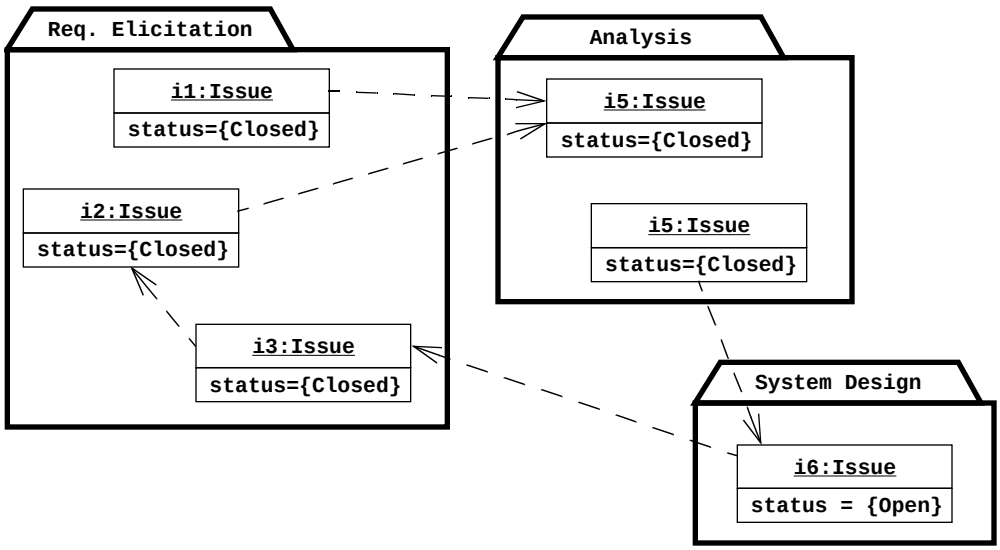

Figure 12-16 The waterfall model as a special case of the issue-based life cycle model (UML object diagram). All issues that belong to the same issue category are contained in the same UML package. In the project status shown in the figure, all the requirements elicitation and analysis issues have been closed; that is, the requirements elicitation and analysis activities have been completed.

In Boehm's spiral model, risks correspond to issues that are evaluated and reopened at the beginning of each round. Issues are resolved in the order of their priority as defined during risk analysis. Note, however, that "issue" is a more general concept than "risk". For example, the issue "Which access control model should we use?" is a design problem, not a risk.

In the general case (Figure 12-17), all activities still may have open issues associated with them, which means that all activities need to be managed concurrently. The goal of the project manager is to keep the number of open issues small and manageable without imposing time or activity-based constraints on the resolution of issues. Using issues and their dependencies for managing the life cycle activities allows all life cycle activities to proceed concurrently.

Software Life Cycle Page 486 Thursday, September 2, 1999 12:33 PM

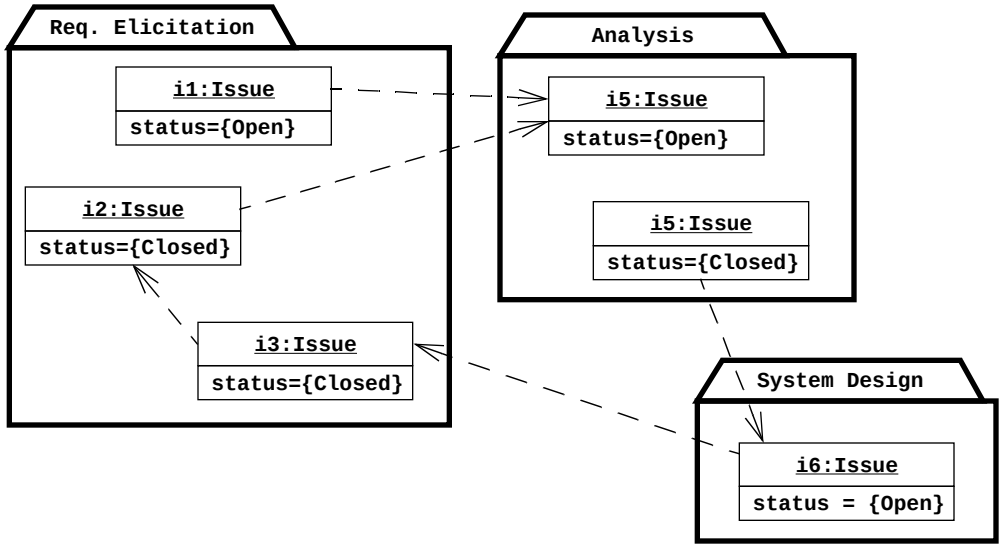

Figure 12-17 In a complex project state, all activities can still have some open issues, which means that all activities need to be managed concurrently.

12.5 Managing activities and products

In this section we describe the process of life cycle modeling for two example projects. The first example project, a four month, single site, prototype development, can be used for a single semester project course. The second example project, an eight month, two site prototype development, can be used for a two semester project course where each semester occurs at a different location.

12.5.1 Example project 1: 4-month, 1-site prototype development

Project goals and constraints

One goal of the project is to develop a demonstration prototype of a software system. A second goal of the project is to expose its participants to state-of-the-art software engineering methods and tools. It has forty participants, most of which have not worked together before. The developers and technical writers involved in the project have strong technical background in writing programs and documentation but no large scale development experience. The application domain is new to all the participants. The delivery of the prototype must occur within four months. Given the short time span, flat staffing must be used.

Activities

We select a life cycle model consisting of three phases. During the first phase, called pre-development, preliminary requirements are formulated and an initial software architecture is

Software Life Cycle Page 487 Thursday, September 2, 1999 12:33 PM

developed for the purpose of allocating resources for the rest of the project. Infrastructure choices are also made during the preliminary phase. Only a subset of the project participants–the client, project manager and coaches–are involved. During the second phase, called development, a system exhibiting the core requirements is constructed following a waterfall model. All developers have been assigned to a team and are actively working towards the construction of the system. The goal of the development phase is to validate the architecture of the system and expose the participants to all aspects of the life cycle. During the third phase, called post-development, the optional functionality is added to the system. The goal of the third phase is to extend the functionality of the system while controlling risks. Functionality is added at all levels of abstractions: The requirements, design, implementation, and test cases are developed incrementally. The rationale for this approach is to deliver a functional system on time as opposed to delivering a complete system late. Table 12-9 describes each of these three phases in terms of objectives and IEEE 1074 activities and processes.

Table 12-9 Activities for example project 1.

Phase	Purpose	Activities	Corresponding IEEE 1074 activities
Pre-development	Initiate project	Project Initiation Concept Exploration Top-level Design	3.1. Project Initiation 4.1 Concept Exploration 4.2 System Allocation
Development	Validate architecture Train participants Demonstrate core requirements Demonstrate feasibility	Project Management Requirements Elicitation Analysis Analysis Review System Design System Design Review Object Design Object Design Review Implementation Unit Testing System Integration Testing System Testing (alpha test) Configuration Management Tutorials	3.2 Project Monitor & Control 5.1 Requirements 5.2 Design 5.2.7 Perform Detailed Design 5.3 Implementation 7.1 Verification & Validation 7.2 Software Configuration Management 7.4 Training
Post-development	Demonstrate optional requirements Deliver prototype Client acceptance	Project Management Refinement Install Software Client Acceptance Test Field Testing (beta test) Configuration Management	3.2 Project Monitor & Control 6.3 Maintenance 6.1.5 Install Software 6.1.6 Accept Software 7.1 Verification & Validation 7.2 Software Configuration Management

Software Life Cycle Page 488 Thursday, September 2, 1999 12:33 PM

Note that this life cycle is not complete, it does not include operation and support activities given that the goal of the example project is to develop a demonstration prototype.

Work products

The project produces three kinds of information: *system information*, *management information*, and *rationale*. This information is represented by three kinds of models: Task models represent management information, system models describe the system (requirements, analysis, design, implementation), and issue models represent the rationale behind design and management decisions.

Task models are detailed, project specific, process models. They are constructed during project initiation by management and revised throughout the project. They are documented in the *Project Agreement*, *Software Project Management Plan*, and the *Top-level Design* document (Table 12-10). We describe task models in Chapter 11, *Project Management.*

Table 12-10 Management documents

Document	Purpose	Produced by
Problem Statement	Describes the needs, visionary scenarios, current system, target system, requirements and constraints	Concept Exploration
Initial Schedule	Describes major milestones.	Project Initiation
Top-level Design	Describes preliminary system architecture, teams, constraints, communication infrastructure.	Top-level Design
Software Project Management Plan	Controlling document for managing the project	Project Initiation Project Monitor and Control
Software Configuration Management Plan	Controlling document for software configuration management activities	Software Configuration Management

System models represent the system at different levels of abstraction and from different perspectives. The use case model describes the system from the user's point of view. The analysis model describes the application objects that the system manipulates. The system and object design models represent the solution objects. The use case and analysis models are documented in the *Requirements Analysis Document* (Table 12-11). The system design model is documented in the *System Design Document*. The object design model and sequence diagrams are documented in the *Object Design Document*. We describe the *System Design Document* and the *Object Design Document* in Chapters 6 and 7, respectively.

Software Life Cycle Page 489 Thursday, September 2, 1999 12:33 PM

Table 12-11 System documents

Document	Purpose	Produced by
Requirements Analysis Document (RAD)	Describes the functional and global requirements of the system as well as 4 models - the use case model, the object model, the functional model and the dynamic model.	Requirements
User Manual	Describes the use of the system often in form of a tutorial	Requirements
System Design Document (SDD)	Describes the design goals, trade-offs made between design goals, the high-level decomposition of the system, concurrency identification, hardware/software platforms, data management, global resource handling, software control implementation and boundary conditions.	System Design
Object Design Document (ODD)	Describes the system in terms of refined object models, in particular the chosen data structures and algorithms as well as full signatures for all public methods. this document results in the detailed specification of each class used by the programmers during the implementation phase.	Object Design
Test Manual	Describes testing strategy, the unit and system tests performed on the system along with expected and actual results.	Testing
Administrator Manual	Describes the administrative procedures to install, operate and bring down the system. Also contains a list of error codes, failure and termination conditions.	Install Software

The rationale of the system is represented with an issue model. Developers focus on the system, argue about different alternatives, have different design goals or base many decisions on prior experience. The motivation behind capturing the rationale is to facilitate communication and make design goals and constraints explicit throughout the project. System goals and constraints are described in the system design document. Issues are described in the *Issues Document* (Table 12-12). We describe issue models in Chapter 8, *Rationale Management.*

Table 12-12 Rationale documents

Document	Purpose	Produced by
Issues Document (ID)	Describes the open issues, possible options, arguments, and their resolution	All processes

Software Life Cycle Page 490 Thursday, September 2, 1999 12:33 PM

12.5.2 Example project 2: 8-month, 2-site prototype development

Project goals and constraints

In this example, we describe a pilot project with two development phases occurring at two sites. The end of the first development phase overlaps with the beginning of the second phase, allowing some knowledge transfer to occur between both sites. Again, most participants are not involved with the pre-development and the post-development phase. Participants at one site have not yet worked with the participants of the other site prior to this project. The delivery of a prototype system must occur within eight months, with an intermediate demonstration of the prototype occurring at the end of the first four months.

Activities

We refine the life cycle of the first example project to include a second development phase and activities focused on transferring knowledge from one site to the other (Table 12-13). Both development phases are structured in the same way, such that the same documents, methods, and procedures can be used. The second development phase is an improvement iteration that produces a refined version of the *Requirements Analysis Document*, the *System Design Document*, the *Object Design Document*, and the prototype system. An additional work product, called the *Inventory Analysis Document*, is produced at the beginning of the second development phase to describe the current status of the system. It includes a list of components, documents, and models that have been constructed in the first phase and a list of issues describing problems in those items that need to be addressed.

Knowledge transfer between sites

Distributed software development can have several advantages. For example, a project can leverage off specific combinations of skills without relocating participants within the organization. Often, the system will also be deployed at different sites and the project manager may want developers to be geographically close to several user communities. Distribution, however, introduces new challenges, such as communicating asynchronously and transferring knowledge from one group of participants to another.

In example project 2, knowledge transfer needs to occur at the end of the first development phase and at the beginning of the second development phase. Since there is minimal overlap between both phases, participants of different sites do not know each other. Consequently, opportunities for spontaneous exchanges do not exist. Instead, we formalize this knowledge transfer with the Inventory Analysis activity. Participants from the second site first become familiar with the system by attending via video conference one or more reviews, such as the object design review or the client acceptance test. Then, participants of the second site reverse engineer the first prototype system to find inconsistencies and omissions with its accompanying documentation. Some of the participants of the first site stay with the project temporarily to wrap up the first development phase and answer questions from participants of the second site. The result of the Inventory Analysis is the *Inventory Analysis Document* describing the components of the first prototype and their status.

Software Life Cycle Page 491 Thursday, September 2, 1999 12:33 PM

Table 12-13 Activities for example project 2. Differences from example 1 highlighted in *italics*.

Phase	Purpose	Activities	IEEE 1074 activities
Pre-development phase	Initiate project	Project Initiation Concept Exploration Top-level design	3.1. Project Initiation 4.1 Concept Exploration 4.2 System Allocation
First development phase	Validate architecture Train participants of first site Demonstrate core requirements Demonstrate feasibility	Project Management Requirements elicitation Analysis System Design Object Design Implementation Unit Testing System Integration Testing System Testing (alpha test) Configuration Management Lectures & Tutorials	3.2 Project Monitor & Control 5.1 Requirements 5.2 Design 5.2.7 Perform Detailed Design 5.3 Implementation 7.1 Verification & Validation 7.2 Software Configuration Management 7.4 Training
Second development phase	*Improve architecture* *Train participants of second site* *Demonstrate extensibility* *Demonstrate desirable requirements*	*Inventory analysis* *Inventory review* *Project Management* *Requirements Elicitation* *Analysis* *Requirements Analysis Review* *System Design* *System Design Review* *Object Design* *Object Design Review* *Implementation* *Unit Testing* *System Integration Testing* *System Testing (alpha test)* *Configuration Management* *Tutorials*	*3.2 Project Monitor & Control* *5.1 Requirements* *5.2 Design* *5.2.7 Perform Detailed Design* *5.3 Implementation* *7.1 Verification & Validation* *7.2 Software Configuration Management* *7.4 Training*
Post-development phase	Demonstrate complete functionality Deliver prototype Client acceptance	Project Management Refinement Install Software Client Acceptance Test Field Testing (beta test) Configuration Management	3.2 Project Monitor & Control 6.3 Maintenance6.1.5 Install Software 6.1.6 Accept Software 7.1 Verification & Validation 7.2 Software Configuration Management

Software Life Cycle Page 492 Thursday, September 2, 1999 12:33 PM

The amount of overlap between both phases presents a critical project management trade-off. A short overlap prevents sufficient knowledge to be transferred. Participants from the second site will then have to reengineer most of the knowledge from work products only. The longer the overlap, the more opportunities there will be for knowledge transfer in the form of targeted questions and answers. If the second development phase starts too early, however, participants of the second site will interfere with the participants of the first site who are still in the process of creating the knowledge being transferred.

Documents

The documents created by this example are the same as in example project 1, in addition to the *Inventory Analysis Document* produced during Inventory Analysis (Table 12-14). The main different between these projects is that the major technical documents (i.e., *Requirements Analysis Document*, *System Design Document, Object Design Document*, *Issue Document*) are revised substantially during the second development phase, to account for the lessons learned during the first development phase.

Table 12-14 Inventory documents

Document	Purpose	Produced by
Inventory Analysis Document	Describes the components and models that have been realized, their status, and the problems associated with them.	Inventory Analysis

12.6 Exercises

1. Assume you have to build a CASE tool for the IEEE 1074 standard life cycle model. Define the actors and use cases for the Design Process.
2. Adapt Figures 12-1, 12-3, and 12-4 to produce an integrated UML class diagram representing the activities and workproducts discussed in this chapter.
3. Assume that the waterfall model shown in Figure 12-8 has been derived from the IEEE standard model in Figure 12-7 during the activity Life Cycle Modeling. What processes and activities have been omitted in the waterfall model?
4. Map the names of the activities in the DOD 2167A standard in Figure 12-9 on page 475 to the names of activities in the IEEE 1074 standard.
5. Redraw Boehm's spiral model in Figure 12-11 as a UML activity diagram. Compare the readability of the original figure with the activity diagram.
6. Draw a UML activity diagram describing the dependency between activities for a life cycle in which requirements, design, implementation, test, and maintenance occur concurrently. (This is called an evolutionary life cycle).
7. Describe how testing activities can be initiated well before implementation activities. Explain why this is desirable.

Software Life Cycle Page 493 Thursday, September 2, 1999 12:33 PM

8. Assume you are part of the IEEE committee that will revise the IEEE 1074 standard. You have been assigned the task of modeling communication as an explicit integral process. Make a case for the activities that would belong to this process.

References

[Ambler, 1998] S. W. Ambler, *Process Patterns: Building Large-Scale Systems Using Object Technology*. Cambridge University Press, New York, 1998.

[Boehm, 1987] B. Boehm, "A spiral model of software development and enhancement," *Software Engineering Project Management*. pp. 128-142, 1987.

[Humphrey, 1989] W. Humphrey, *Managing the Software Process*. Addison-Wesley, Reading, MA, 1989.

[IEEE, 1997] *IEEE Standards Collection Software Engineering*. IEEE, Piscataway, NJ, 1997.

[IEEE Std. 1074-1995] *IEEE Standard for Developing Software Life Cycle Processes*, IEEE Computer Society, New York, 1995, in [IEEE 1997].

[Jacobson et al., 1999] I. Jacobson, G. Booch, & J. Rumbaugh, *The Unified Software Development Process*. Addison-Wesley, Reading, MA, 1999.

[Jensen & Tonies, 1979] R. W. Jensen & C. C. Tonies, *Software Engineering*, Prentice Hall, Englewood Cliffs, NJ, 1979.

[Paulk et al., 1995] M. C. Paulk, C. V. Weber, & B. Curtis (eds.), *The Capability Maturity Mode: Guidelines for Improving the Software Process*. Addison-Wesley, Reading, MA, 1995.

[Rational, 1998] *Rationale Rose 98: Using Rose*. Rational Software Corp., Cupertino, CA, 1998.

[Rowen, 1990] R. B. Rowen, "Software project management under incomplete and ambiguous specifications," *IEEE Transactions on Engineering Management*, vol. 37, no. 1, 1990.

[Royse, 1970] W. W. Royse, "Managing the development of large software systems," in *Tutorial: Software Engineering Project Management*, IEEE Computer Society, Washington, DC, pp. 118–127, 1970.